

Mobile Application Development with Flutter

Lecture 47. Provider State Management in Flutter

Big Picture

- With setState, state lives inside a widget
- With Provider, state lives outside the widget and can be shared anywhere

Core Idea

Provider separates:

- UI (widgets)
- Logic (state + functions)

STEP 1: Add Provider Package

- Inside YAML file add
- dependencies:
 provider: ^6.1.5+1
- Flutter doesn't include Provider by default—we install it to manage state
- Alternate way is to open terminal and run these commands
 - flutter pub add provider
 - flutter pub get

STEP 2: Create Provider Class

- Create a new file called “counter_provider.dart”

```
import 'package:flutter/material.dart';

class CounterProvider extends ChangeNotifier {
  int count = 0;

  void increment() {
    count++;
    notifyListeners();
  }
}
```

Explanation

- `ChangeNotifier`
 - This tells Flutter: this class can notify when data changes
- `count`
 - This is our state (data)
- `Increment()`
 - Business logic (what to do when user clicks button)
- `notifyListeners()`
 - This tells all listening widgets: rebuild yourself
 - Without `notifyListeners`, UI will never update

STEP 3: Provide the State

- In main.dart

```
runApp(  
  ChangeNotifierProvider(  
    create: (context) => CounterProvider(),  
    child: MyApp(),  
  ),  
);
```


Explanation

- ChangeNotifierProvider
 - This makes our provider available to the entire app
- create
 - Creates an instance of CounterProvider
- Why wrap at top?
 - So ALL screens can access it

STEP 4: Access Data in UI

- User consumer

```
Consumer<CounterProvider>(  
  builder: (context, provider, child) {  
    return Text(  
      "Counter: ${provider.count}",  
      style: TextStyle(fontSize: 28),  
    );  
  },  
)
```


Explanation

- Consumer
 - Listens to provider changes and rebuilds only this part
- provider.count
 - Accessing state from provider

Prepared by Ehtisham Rasheed

STEP 5: Update Data (Trigger Change)

```
context.read<CounterProvider>().increment();
```

- read()
 - Access provider WITHOUT listening
- Why not watch?
 - Because button doesn't need rebuild

Flow

👉 Button Click

- increment()
- count changes
- notifyListeners()
- Consumer rebuilds